PROGRAMOZÁSI TECHNOLÓGIA BEADANDÓ

MINTADOKUMENTÁCIÓ

[Pásztor Alexa Zorka]

[IOSDX4]

FELADAT LEÍRÁSA

Közös elvárás a megoldásoknál, hogy gyűjteményben tároljuk az azonos ősosztályból származtatott objektumait. Az objektumok feldolgozása során használjunk **foreach** szerkezetet. Hibás adatok megadása esetén a program dobjon kivételt, amit kezeljünk is le.

Szimuláljuk az alábbi egyszerűsített Capitaly társasjátékot! Adott néhány eltérő stratégiájú játékos és egy körpálya, amelyen különféle mezők sorakoznak egymás után. A pályát körbe-körbe újra és újra bejárják a játékosok úgy, hogy egy kockával dobva mindig annyit lépnek, amennyit a kocka mutat. A mezők három félék lehetnek: ingatlanok, szolgáltatások és szerencse mezők. Az ingatlant meg lehet vásárolni 1000 Petákért, majd újra rálépve házat is lehet rá építeni 4000 Petákért. Ha ezután más játékos erre a mezőre lép, akkor a mező tulajdonosának fizet: ha még nincs rajta ház, akkor 500 Petákot, ha van rajta ház, akkor 2000 Petákot. A szolgáltatás mezőre lépve a banknak kell befizetni a mező paramétereként megadott összeget. A szerencse mezőre lépve a mező paramétereként megadott összegű pénzt kap a játékos. Háromféle stratégiájú játékos vesz részt a játékban.

Kezdetben mindenki kap egy induló tőkét (10000 Peták), majd a „mohó” játékos, ha egy még gazdátlan ingatlan mezőjére lépett, vagy övé az ingatlan, de még nincs rajta ház, továbbá van elég tőkéje, akkor vásárol. Az „óvatos” játékos egy körben csak a tőkéjének a felét vásárolja el, a „taktikus” játékos minden második vásárlási lehetőséget kihagyja. Ha egy játékosnak fizetnie kell, de nincs elégendő pénze, akkor kiesik a játékból, házai elvesznek, ingatlanjai megvásárolhatókká válnak.

A játék paramétereit egy szövegfájlból olvassuk be. Ez megadja a pálya hosszát, majd a pálya egyes mezőit. Minden mezőről megadjuk annak típusát, illetve, ha szolgáltatás vagy szerencse mező, akkor annak pénzdíját. Ezt követően a fájl megmutatja a játékosok számát, majd sorban minden játékos nevét és stratégiáját. A tesztelhetőséghez fel kell készíteni a megoldó programot olyan szövegfájl feldolgozására is, amely előre rögzített módon tartalmazza a kockadobások eredményét.

Írjuk ki, hogy adott számú kör után hogyan állnak (mennyi a tőkéjük, milyen ingatlanokat birtokolnak) a versenyzők!

FELADAT ÉRTELMEZÉSE

A feladat azt várja el, hogy készítsünk egy olyan programot, ami egy fájlban előre megadott adatok alapján szimulál egy játékot, szintén megadott dobások alapján bizonyos számú körön keresztül, majd lefutás után kiírja a játékosok állását.

PROGRAM TERVEZETE

A program több egymásra támaszkodó/ épülő osztályból áll. A program feladata egy társasjáték működésének leszimulálása, ahol a játékosok különböző stratégiák szerint játszanak. Pontosan 3 féle stratégia közül választanak és birkóznak meg a játéktáblával, ami szintén 3 különbféle mezőből áll.

A játékosok ősosztálya a **Player** osztály. Ez az *absztrakt* osztály tartalmazza a játékosok alap adatait, vagyis: név, pénz, pozíciójuk a táblán és a játék alatt vásárolt ingatlanjaikat. A stratégiájukhoz tartozó vásárlásaikat pedig a leszármazott osztályok valósítják meg.

Ezek a leszármazottak a: **Greedy**, **Careful** és **Tactical.** A 3 stratégia 3 különbféle ingatlanvásárlási technikát jelent. A buy() metódus minden játkosnál a stratégiája szerint hajtja végre az ingatlanvásárlást. Mindezt egy override segítségével teszi.

A különböző mezők absztrakt ősosztálya a **Field** osztály, leszármazottai pedig az **Estate, Service** és **Luck** osztályok. Az **Estate** osztály a megvásárolható ingatlanokat fogalja magában. Ezek azok a mezők, amiket a játékosok megvásárolhatnak, illetve házat építhetnek rá később. A **Luck** osztály mezői pénzjutalomban részesítik a játékost, míg a **Service** osztály mezőire lépve pénzbüntetést kap a játékos.

A mezőket a **Board** osztály kezeli egy Field típusú *ArrayList*-ben tárolva, így ugyanabban a gyűjteményben tárolható az Estate, Luck és a Service osztályok mezői is.

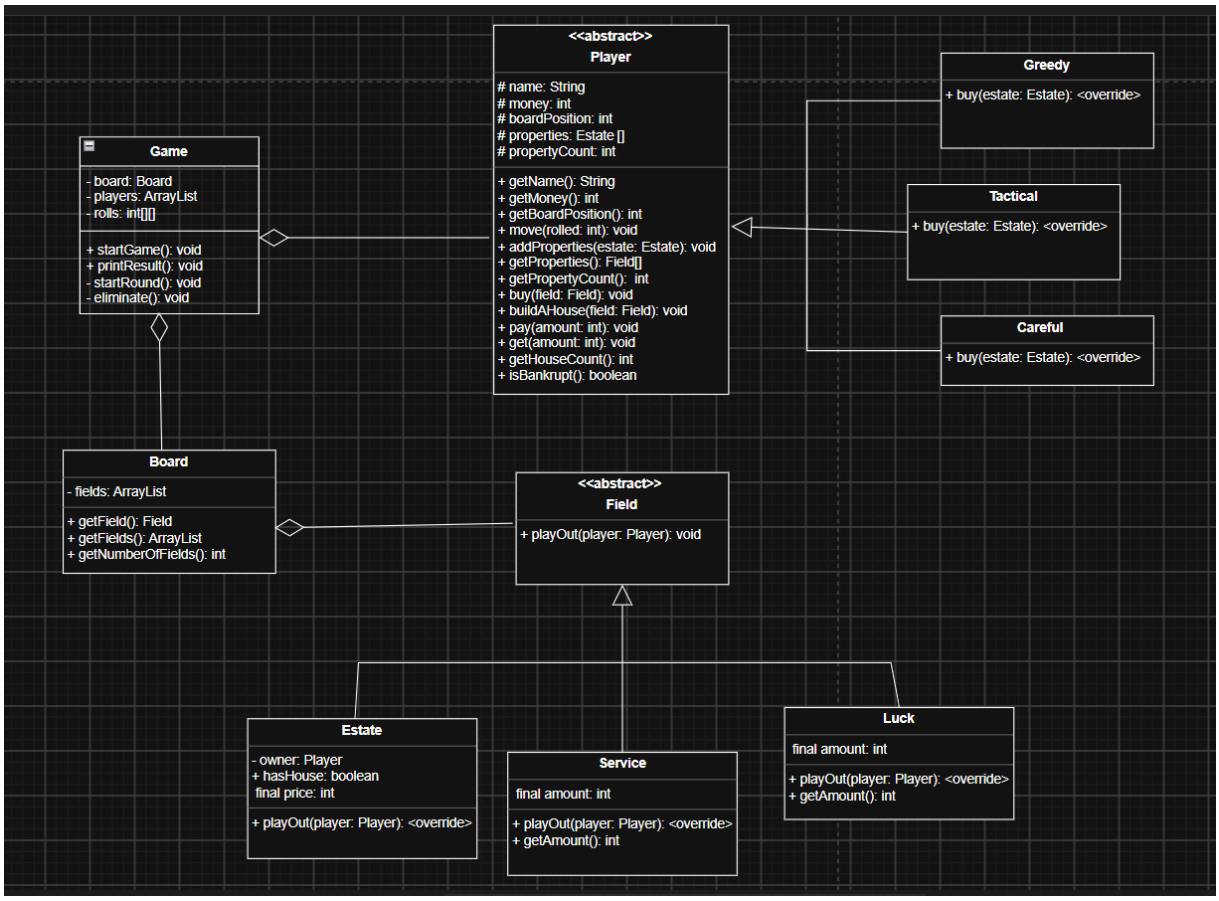
A játékosokat szintén egy Player típusú *ArrayList*-ben tárolja a program, ugyanezt az előnyt kihasználva.

Magáért a játék működésért a **Game** osztály felel. Ez az osztály tárolja a játéktáblát, a játékosokat, valamint a dobásokat. A **startGame()** metódus végrehajtja a köröket, minden körben játékosoként. Végrehajtja a dobásukat, illetve az adott játékos stratégiájához tartozó **playOut()** metódust.

A fájlfeldolgozásért a **FileReader** osztály felelős. Beolvassa sorban a mezők számát, illetve típusait, a játékosok számát, majd a nevüket és a stratégiájukat. A mezők és a játékosok esetében is létrehozza a megfelelő objektumokat, majd a fájl többi részéből a játékosok dobását körönként.

A program fő eleme talán a **Main** osztály. Az osztály **main**() metódusa bekéri a fájlt, majd beolvastatja az adatokat, létrehozza a játékot és meghívja a **startGame**() metódust. A megadott körök lefuttatása után a program kiírja a játékosok eredményeit, vagyis a megmaradt pénzüket, valamint a birtokolt ingatlanok és házak számát.

OSZTÁLYDIAGRAM



FONTOSABB METÓDUSOK

buy(): Ez a metódus adja vissza minden származtatott osztályban az adott stratégiához tartozó vásárlást. Override szükséges minden implementációkor.

startGame(): Ez a metódus az, ami meghíváskor lefuttatja magát a játék szimulációt

playOut(): Ez a metódus adja vissza minden származtatott osztályban az adott mezőre lépéskor történő utasítást. Override szükséges minden implementációkor.

PROGRAM FUTÁS KÖZBEN

```
"C:\Program Files\Java\jdk-25.0.4.1\bin\java.exe" "-javaa
What's the name of your file?: game1.txt
------- Players at the certain round -------
Anna has 9600 Petak
Properties: 0
Houses: 0

----------------------------------------
Betty has 4500 Petak
Properties: 2
Houses: 1

----------------------------------------
Carmilla has 8000 Petak
Properties: 1
Houses: 0

----------------------------------------

Process finished with exit code 0
```

```
"C:\Program Files\Java\jdk-25.0.4.1\bin\java.exe" "-java
What's the name of your file?: game3.txt
File not found!

Process finished with exit code 0
```

```
"C:\Program Files\Java\jdk-25.0.4.1\bin\java.exe" "-javaa
What's the name of your file?: gameempty.txt
That is an empty file mi amigo

Process finished with exit code 0
```

```
"C:\Program Files\Java\jdk-25.0.4.1\bin\java.exe" "-javaa
What's the name of your file?: gameallhave1.txt
------- Players at the certain round -------
Anna has 5000 Petak
Properties: 1
Houses: 1

----------------------------------------
Betty has 5000 Petak
Properties: 1
Houses: 1

----------------------------------------
Carmilla has 5000 Petak
Properties: 1
Houses: 1

----------------------------------------

Process finished with exit code 0
```

FELHASZNÁLÓI TÖRTÉNETEK

AS A felhasználó

I WANT TO megtudni, melyik játékos áll nyerésre

1	GIVEN	megadjuk futáskor a fájl nevét, game1.txt
	WHEN	betöltjük a fájlban található adatokat
	THEN	megkapjuk az adott fájlban levő körök utáni állást

2	GIVEN	megadjuk futáskor a fájl nevét, game2.txt
	WHEN	betöltjük a fájlban található adatokat
	THEN	megkapjuk az adott fájlban levő körök utáni állást

3	GIVEN	megadjuk futáskor a fájl nevét, game3.txt
	WHEN	a fájl nem létezik
	THEN	a program kilép és jelzi, hogy ez a fájl nem létezik

4	GIVEN	nem adunk fájl nevet
	WHEN	nem a kap a beolvasás fájlt
	THEN	a program el se indul, továbbra is fájl nevet vár

5	GIVEN	megadjuk futáskor a fájl nevét, game1
	WHEN	lemaradt a .txt kiterjesztés
	THEN	megkapjuk, hogy nincs ilyen fájl

6	GIVEN	megadjuk futáskor a fájl nevét, wrongfieldcount.txt
	WHEN	a fájlban hiányzik 1 mező
	THEN	a program kilép InputMismatchException- nel

7	GIVEN	megadjuk futáskor a fájl nevét, hasnorolls.txt
	WHEN	betöltjük a fájlban található adatokat
	THEN	a program lefut a játékos adatok módosítása nélkül

8	GIVEN	megadjuk futáskor a fájl nevét, minusplayers.txt
	WHEN	betöltjük a fájlban található adatokat, de -4 játékos van megadva
	THEN	kiírja, a felvezető üzenetet, de semmi mást

9	GIVEN	megadjuk futáskor a fájl nevét, gameempty.txt
	WHEN	a fájl nem tartalmaz adatokat
	THEN	a program jelzi, hogy ez a fájl üres

10	GIVEN	megadjuk futáskor a fájl nevét, gameallhave1.txt
	WHEN	betöltjük a fájlban található adatokat
	THEN	megkapjuk, hogy mindenkinek van legalább 1 ingatlanja és legalább 1 háza